

Indian Institute of Information Technology Bhopal



Digital Image Processing

Level1: Blurring Image



विद्या ददाति विनयं

Submitted by:	Guided by:
Aryan Chouhan and Yuvraj Singh	Dr. Supriya Agrawal
23U03121 and 23U03086	Prof IIIT Bhopal

L1 Project: Blurring Image using Averaging and Gaussian Algorithm

AIM

To study and implement image blurring techniques using Averaging (Mean) and Gaussian algorithms in Python, understanding their concepts, mathematical principles, and practical applications in digital image processing.

Theory

Image blurring, also known as smoothing, is a fundamental operation in digital image processing that reduces the sharpness and detail of an image. It is commonly used to remove noise, reduce image details, and prepare images for further processing like edge detection or segmentation.

1. Averaging Blur (Mean Filter):

- The averaging filter replaces each pixel value in an image with the mean (average) value of its neighboring pixels.
- This method uses a kernel filled with equal weights and performs convolution with the image.
- Formula:
$$I'(x, y) = (1 / (m \times n)) * \sum \sum I(x+i, y+j)$$
- It smoothens the image but tends to blur edges.

2. Gaussian Blur:

- The Gaussian blur uses a Gaussian function to assign different weights to pixels in the kernel.
- Central pixels get higher weights, resulting in smoother and more natural blurring.
- Formula:
$$G(x, y) = (1 / (2\pi\sigma^2)) * \exp(-(x^2 + y^2) / (2\sigma^2))$$
- This filter is widely used for noise reduction and as a preprocessing step before edge detection.

Code

Below is the implementation of Averaging and Gaussian Blur algorithms using Python, NumPy, OpenCV, and Matplotlib.

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

def averaging_blur(img: np.ndarray, kernel_size: int = 3) -> np.ndarray:
    if kernel_size % 2 == 0:
        raise ValueError("kernel_size must be odd.")
    kernel = np.ones((kernel_size, kernel_size), dtype=np.float32) /
(kernel_size * kernel_size)
    k = kernel_size // 2
    if len(img.shape) == 3:
        channels = cv.split(img)
```

```

        blurred_channels = [averaging_blur(ch, kernel_size) for ch in
channels]
        return cv.merge(blurred_channels)
    padded = cv.copyMakeBorder(img, k, k, k, k,
borderType=cv.BORDER_REPLICATE)
    blurred_img = np.zeros_like(img, dtype=np.float32)
    rows, cols = img.shape
    for i in range(rows):
        for j in range(cols):
            window = padded[i:i+kernel_size, j:j+kernel_size]
            blurred_img[i, j] = np.sum(window * kernel)
    return np.clip(blurred_img, 0, 255).astype(np.uint8)

def gaussian_kernel(kernel_size: int, sigma: float) -> np.ndarray:
    k = kernel_size // 2
    x, y = np.mgrid[-k:k+1, -k:k+1]
    kernel = np.exp(-(x**2 + y**2) / (2 * sigma**2))
    kernel = kernel / (2 * np.pi * sigma**2)
    return kernel / np.sum(kernel)

def gaussian_blur(img: np.ndarray, kernel_size: int = 3, sigma: float =
1.0) -> np.ndarray:
    if kernel_size % 2 == 0:
        raise ValueError("kernel_size must be odd.")
    kernel = gaussian_kernel(kernel_size, sigma)
    k = kernel_size // 2
    if len(img.shape) == 3:
        channels = cv.split(img)
        blurred_channels = [gaussian_blur(ch, kernel_size, sigma) for ch in
channels]
        return cv.merge(blurred_channels)
    padded = cv.copyMakeBorder(img, k, k, k, k,
borderType=cv.BORDER_REPLICATE)
    blurred_img = np.zeros_like(img, dtype=np.float32)
    rows, cols = img.shape
    for i in range(rows):
        for j in range(cols):
            window = padded[i:i+kernel_size, j:j+kernel_size]
            blurred_img[i, j] = np.sum(window * kernel)
    return np.clip(blurred_img, 0, 255).astype(np.uint8)

if __name__ == '__main__':
    path_image = 'input.jpg'
    img = cv.imread(path_image)

```

```
averaging_blured_img = averaging_blur(img)  
gaussian_blured_img = gaussian_blur(img)  
cv.imwrite('OutputAVG.jpg', averaging_blured_img)  
cv.imwrite('OutputGauss.jpg', gaussian_blured_img)
```

Input:



Output:

1. Averaging Blur



2. Gaussian Blur



Result

Both algorithms successfully blurred the input image. The Averaging Blur resulted in uniform smoothing but with noticeable edge loss, while the Gaussian Blur produced smoother and more natural-looking results due to weighted averaging.

The output images (OutputAVG.jpg and OutputGauss.jpg) demonstrate the difference between both blurring techniques.

Learnings

1. Understood the concept of convolution and kernel operations in image processing.
2. Learned how different kernel weight distributions affect image sharpness and smoothness.
3. Averaging blur is simple but less effective at preserving edges.
4. Gaussian blur provides a more natural smoothing and is widely used in computer vision tasks.
5. Gained practical experience with OpenCV, NumPy, and Matplotlib libraries for image processing.